

## Game — Debug the Module

A short exercise in using Python's interactive debugger (`pdb`) to find a bug that lives in an **imported module**, not your main script.

---

### The Bug

`main.py` imports `calculate_total` from `shop.py`, but the result is wrong:

```
python

# main.py
from shop import calculate_total

cart = [("apple", 2, 1.50),
        ("bread", 1, 3.00),
        ("milk", 3, 2.25)]

print(calculate_total(cart))

# Expected: 12.75 → Got: 6.75 🐞
```

The bug is somewhere inside `shop.py`:

```
python

# shop.py
def calculate_total(cart):
    total = 0
    for name, qty, price in cart:
        total += price
    return total

# 🐛 Bug is somewhere in here!
```

---

### Your Mission

1. `main.py` imports `calculate_total` from `shop.py` — but the result is wrong.
2. Open `shop.py` and add `breakpoint()` as the **first line inside the `for` loop**.
3. Run `python main.py` from the terminal. Execution pauses inside `shop.py`.

4. At the `(Pdb)` prompt, type `p name`, `p qty`, and `p price` for each item.
5. Type `n` to step forward and watch how `total` changes — is `qty` being used?
6. Find the missing piece, fix the line, remove `breakpoint()`, and re-run!

---

### Cross-File Tip

`pdb` pauses wherever `breakpoint()` is — **even if that's inside an imported module, not your main script**. You don't need to put the breakpoint in `main.py` to debug code that lives elsewhere; the debugger follows execution across file boundaries.

---

## Files

| File                 | Role                                                                |
|----------------------|---------------------------------------------------------------------|
| <code>shop.py</code> | The module containing <code>calculate_total()</code> — and the bug. |
| <code>main.py</code> | Entry point. Imports and calls <code>calculate_total()</code> .     |

## Requirements

- Python 3.7+ (for built-in `breakpoint()` support)

## Running

```
bash

python main.py
```

Once you've added `breakpoint()` to `shop.py` as instructed above, this will drop you into an interactive `(Pdb)` session at that line.

### Useful `pdb` commands for this exercise

| Command                     | Effect                                                                   |
|-----------------------------|--------------------------------------------------------------------------|
| <code>p &lt;expr&gt;</code> | Print the value of <code>&lt;expr&gt;</code> (e.g., <code>p qty</code> ) |
| <code>n</code>              | Next line (step over)                                                    |

---

| Command        | Effect                                       |
|----------------|----------------------------------------------|
| <code>c</code> | Continue execution until the next breakpoint |
| <code>l</code> | List source code around the current line     |
| <code>q</code> | Quit the debugger                            |
|                |                                              |
|                |                                              |

## Solution

<details> <summary>Click to reveal (try it yourself first!)</summary>

The loop unpacks `qty` but never uses it — `total` accumulates `price` alone instead of `price * qty`:

```
python
total += price * qty
```

With the fix: `2×1.50 + 1×3.00 + 3×2.25 = 3.00 + 3.00 + 6.75 = 12.75` 

</details>